

Laboratorio 7 - Esercizi in stile esame

Lo scopo del Laboratorio 7 è di esercitarsi con problemi nello stile e nella dimensione tipici dei temi d'esame del corso. A differenza dei laboratori precedenti, qui non c'è un nuovo argomento: ogni esercizio è un caso d'uso autonomo che richiede di combinare gli strumenti già visti (`Mutex`, `Condvar`, canali, `Arc`, `thread::spawn`, `Drop`, smart pointer, generici, tratti, closure, ...).

Nella cartella `src/` è presente un file sorgente per ciascun esercizio, contenente il codice di partenza e i test unitari da superare. Come per i laboratori precedenti, è possibile eseguire i test con `cargo test <nome_esercizio>` o `cargo test` per eseguirli tutti.

Tutti gli esercizi devono essere **thread-safe** e gestire correttamente il rilascio delle risorse (`Drop`). Le strutture devono comportarsi correttamente anche quando vengono usate in scenari di forte concorrenza, e non devono consumare cicli di CPU in attese attive.

I dettagli del comportamento dei vari metodi dei tratti e/o struct da implementare sono nei rispettivi file sorgente (e.g. `es_2_wait_group.rs`).

Esercizio 1 - AsymmetricRendezvous

In un sistema concorrente è spesso utile far incontrare due thread che giocano ruoli differenti per scambiarsi informazioni di natura diversa. Si realizzi una primitiva di sincronizzazione `AsymmetricRendezvous<Req: Send, Resp: Send>` che modella una singola transazione richiesta/risposta tra un thread "cliente" e un thread "servitore", scambiando dati di tipo diverso nelle due direzioni.

La creazione della coppia di estremi avviene tramite la funzione

Rust

```
pub fn make_rendezvous<Req: Send, Resp: Send>() ->
(ClientEnd<Req, Resp>, ServerEnd<Req, Resp>)
```

I due estremi offrono rispettivamente i seguenti metodi:

Rust

```
impl<Req: Send, Resp: Send> ClientEnd<Req, Resp> {
    pub fn request(self, req: Req) -> Option<Resp>;
}

impl<Req: Send, Resp: Send> ServerEnd<Req, Resp> {
```

```
pub fn accept<F>(&self, handler: F) -> Option<()>
    where F: FnOnce(Req) -> Resp;
}
```

`request(req)` consegna `req` al lato servente e si blocca senza consumare cicli di CPU fino a che il servente non produce la risposta corrispondente, restituendola sotto forma di `Some(resp)`. Il metodo `accept(handler)` si blocca senza consumare cicli di CPU fino a che il cliente non ha consegnato la propria richiesta, quindi invoca `handler` su tale richiesta e consegna al cliente il valore restituito.

Si noti che entrambi i metodi consumano l'estremo su cui sono invocati: il rendezvous è quindi monouso. Se uno dei due estremi viene distrutto senza aver invocato il proprio metodo, l'altro estremo deve sbloccarsi restituendo `None`.

Si implementi tale struttura in linguaggio Rust avendo cura che la sua implementazione sia thread-safe.

Esercizio 2 - WaitGroup

Si implementi in linguaggio Rust la struct `WaitGroup`, una primitiva che permette ad uno o più thread di attendere il completamento di un insieme di operazioni il cui numero non è necessariamente noto al momento della costruzione. A differenza di un classico conteggio alla rovescia, il numero di operazioni da attendere può crescere dinamicamente, anche mentre altri thread sono già bloccati in attesa.

```
Rust
impl WaitGroup {
    pub fn new() -> Self;
    pub fn increment(&self, delta: usize);
    pub fn decrement(&self);
    pub fn wait(&self);
    pub fn wait_timeout(&self, d: Duration) ->
        std::sync::WaitTimeoutResult;
}
```

Una `WaitGroup` appena costruita ha contatore zero, dunque `wait` ritorna immediatamente; solo dopo una chiamata a `increment` il contatore diventa positivo. Si garantisca che un thread che invoca `increment` dopo che il contatore è temporaneamente tornato a zero possa nuovamente bloccare i successivi `wait`. La struttura deve essere condivisibile tra più thread tramite `Arc`.

Esercizio 3 - PriorityChannel

In un sistema di gestione attività si vuole disporre di un canale di comunicazione concorrente in cui i messaggi non vengono consegnati nell'ordine di invio, bensì in ordine di priorità decrescente: un messaggio con priorità più alta scavalca eventuali messaggi a priorità inferiore già presenti nella coda. Si realizzi a tale scopo la struttura `PriorityChannel<T: Send + Ord>`, che offre i seguenti metodi:

```
Rust
impl<T: Send + Ord> PriorityChannel<T> {
    pub fn new(cap: NonZeroUsize) -> Self;
    pub fn send(&self, t: T) -> Option<()>;
    pub fn recv(&self) -> Option<T>;
    pub fn close(&self);
}
```

Si presti particolare attenzione al caso in cui un produttore in attesa di spazio venga sbloccato dalla chiusura del canale, e al caso in cui un consumatore in attesa di un messaggio venga sbloccato dalla chiusura del canale a coda vuota.

Si implementi tale struttura in linguaggio Rust, garantendone la correttezza in presenza di più produttori con un singolo consumatore, senza utilizzare i canali forniti dalla libreria standard o di terze parti.

Esercizio 4 - Watchdog

In molti sistemi è necessario garantire che una determinata attività dia segno di vita ad intervalli regolari: se ciò non accade entro un tempo prestabilito, si presume che l'attività si sia interrotta e si invoca una procedura di emergenza. Si implementi a tale scopo la struttura `Watchdog`, che offre i seguenti metodi:

```
Rust
impl Watchdog {
    pub fn new<F>(timeout: Duration, on_timeout: F) -> Self
        where F: Fn() + Send + 'static;
    pub fn feed(&self) -> bool;
    pub fn is_expired(&self) -> bool;
}
```

Al proprio interno il watchdog incapsula un thread che attende, senza consumare cicli di CPU, lo scadere del tempo residuo: ogni `feed` deve poter accorciare o allungare tale attesa

in modo coerente con la nuova scadenza. Alla distruzione del `Watchdog`, se la callback non è ancora stata invocata, essa deve essere considerata cancellata e il thread interno deve terminare in modo pulito senza eseguirla; la `drop` non deve ritornare al chiamante prima che il thread interno sia terminato.

Si implementi tale struttura in linguaggio Rust, avendo cura di rendere i metodi thread-safe e di evitare corse critiche tra `feed`, scadenza naturale del timer e distruzione.

Esercizio 5 - MemoCache

Si vuole costruire una struttura di memoizzazione concorrente `MemoCache<K, V>` che, dato un valore di chiave `K` ed una funzione di calcolo che produce un valore di tipo `V`, garantisca che la funzione di calcolo venga eseguita una sola volta per ciascun valore distinto di chiave, anche in presenza di più thread che richiedono concorrentemente lo stesso valore.

Rust

```
impl<K: Eq + Hash + Clone + Send + 'static, V: Send + Sync + 'static> MemoCache<K, V> {  
    pub fn new() -> Self;  
    pub fn get_or_compute<F>(&self, key: K, compute: F) -> Arc<V>  
        where F: FnOnce() -> V;  
}
```

Si presti attenzione al fatto che, se il calcolo per una chiave richiede un tempo apprezzabile, il `MemoCache` non deve restare bloccato sotto un lock globale per tutta la durata del calcolo, altrimenti tutte le altre operazioni rimarrebbero in attesa anche se relative a chiavi diverse.

Si implementi tale struttura in linguaggio Rust, garantendone la correttezza e l'efficienza in presenza di richieste concorrenti.

Esercizio 6 - Throttle

In un'applicazione che interagisce con un servizio esterno con limiti di chiamata (rate limit), è necessario assicurarsi che la frequenza con cui una determinata operazione viene invocata non superi un valore prestabilito, indipendentemente dal numero di thread che la richiedono. Si realizzi a tale scopo la struttura `Throttle<F, R>` che incapsula una funzione e ne controlla la frequenza di invocazione globale.

Rust

```
impl<F, R> Throttle<F, R>  
where
```

```

F: Fn() -> R + Send + Sync + 'static,
R: Send + 'static,

{
    pub fn new(f: F, max_calls: NonZeroUsize, window: Duration)
-> Self;
    pub fn call(&self) -> R;
    pub fn try_call(&self) -> Option<R>;
}

```

Si noti che il `Throttle` deve garantire equità tra i thread chiamanti: se più thread sono in attesa contemporaneamente, l'ordine di sblocco deve corrispondere all'ordine di arrivo, evitando situazioni di starvation in cui un thread possa essere indefinitamente scavalcato da chiamanti successivi.

Si noti inoltre che il parametro `window` del costruttore definisce il tempo minimo che deve passare tra una chiamata e l'altra della funzione, senza includere il tempo di *esecuzione* della chiamata stessa. Questo vuol dire che se la `window` è di 1 secondo e `F` ci mette più di 1 secondo a terminare, due chiamate contemporanee ad `F` sono permesse fintanto che sia passato almeno un secondo dall'inizio della prima invocazione.

Si implementi tale struttura in linguaggio Rust.